CAE: Browser-Native Representations for Web Agents

Lexmount Team

Web agents need to repeatedly read page state, choose operations, and interpret their outcomes. Efficient execution requires organizing this information so that the agent can complete tasks at lower model cost. We introduce CAE, a browser-native representation that connects page Content, available Actions, and observed Effects. Constructed from browser layout, paint order, and execution events, CAE organizes rendered content around identified targets and supported operations, linking each action to observed changes in page state. Across 167 matched entry pages, CAE reduces aggregate token count by 26.7% compared with Browser Use DOM outputs. Building on this representation, we design CAE Agent, an LLM-based web agent that maintains a versioned page view and constructs decision contexts from incremental updates and observed Effects. To support evaluation, we construct VerifiedWebTestBench, a collection of 197 real-website tasks with explicit completion checks over independently collected evidence. We use it to compare task success and cost across five agent systems and four LLM backbones. CAE Agent achieves the highest aggregate task success of 85.79% among the evaluated systems, while reducing aggregate cost by 46.18% compared with AgentBrowser.

Date: October 29, 2026

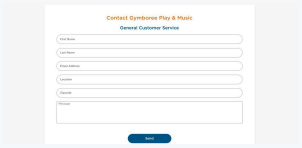
(a) Screenshot

Before



↓ Page transition

After



(b) DOM

Before

```
[222]<a id=nav-GymboFra />
    Become a Franchisee
[226]<a />
    Sign In
[238]<li level=1 />
    [239]<a id=nav-find-location />
        Find a Location
```

↓ New page observation

After

```
Contact Gymboree Play & Music
...
National Service
(415)604-3094
Mon-Fri: 8am to 5pm(PT)
Holidays excluded
Customer Care
```

(c) CAE

Content

```
[6|304|click]Our Company
[6|307|click]Franchise Info
[6|310|click]Job Opportunities
[6|314|click]Customer Care
[6|317|click]Contact Us
[6|320|click]Share Your Feedback
```

Action → click #317

Effect → navigation(settled)

Refreshed Content

```
[2|413|click]Send
[2]National Service
[2](415)604-3094
Mon-Fri: 8am to 5pm(PT)
Holidays excluded
[2]Customer Care
```

Figure 1 Three interface views of a customer-service lookup. Screenshot crops provide visual context. CAE links a target and its advertised action to a typed navigation Effect and refreshed page content.

1 Introduction

Web agents carry out natural-language instructions through sequences of interactions with websites [8, 13, 51]. Each decision requires a view of the current page, a way to select an operation, and feedback on what happened. An agent may revisit the same page during a task after entering a value, changing a filter, or opening a result. These steps combine persistent page structure with local changes, while page observations and interaction history accumulate in the model's context. The practical objective is to support these decisions economically while preserving the information needed to complete the task. This motivates joint evaluation of task success and model cost [16].

The browser interface shapes this recurring context cost by determining what information reaches the agent and how it is organized. DOM and accessibility-tree serializations organize page structure, while screenshots expose rendered appearance [21, 23, 51]. Pruning and selection can reduce the material presented at each step [18, 43, 47]. For continued interaction, an agent also needs to relate that material to operable targets and to changes observed after its actions. We therefore study a representation that connects page content to available operations and observed Effects throughout the interaction loop.

The browser already maintains the information needed to construct this representation. Rendering associates document nodes with layout geometry and paint order, and execution produces observable state changes and navigation events. From these signals, we construct CAE, a browser-native representation that connects page Content, available Actions, and observed Effects. Content presents rendered page content in a hierarchy with identified controls and their supported operations. Action dispatches an operation to a listed target. Effect associates the operation with browser-visible changes and refreshed state. This connection gives the agent a structured basis for choosing its next action. Figure 1 compares screenshot, DOM, and CAE views of a customer-service lookup. The screenshot supplies visual context, and the DOM view lists page elements with their indices. The CAE view links the customer-service target to its advertised click operation, then pairs the navigation Effect with refreshed contact-page content. CAE constructs these links directly from browser state without a language-model call, giving the agent a consistent structure for successive decisions. We compare CAE, full-page Browser Use DOM [4], and AgentOccam [43] representations from the same browser states across 167 matched entry pages. CAE reduces aggregate token count by 26.7% relative to full-page Browser Use DOM outputs and by 46.5% relative to AgentOccam's pruned accessibility-tree outputs.

Building on CAE, we design CAE Agent, an LLM-based web agent that maintains a versioned page view across successive decisions. The LLM selects operations from Content. The harness executes them in order and uses Effect to update its page view. Small changes enter the decision context as versioned patches, accompanied by a chronological record of actions and observed Effects. Stable instructions and earlier context precede changing results, preserving reusable request prefixes within a task. When a page changes substantially, the harness establishes a fresh baseline. This organization lets the agent use compact updates within a page and full views after page transitions.

Evaluating this agent and its underlying representation requires real-website tasks with repeatable completion checks. Self-hosted environments such as WebArena support programmatic evaluation on controlled websites [51]. Live-site benchmarks such as WebVoyager evaluate agents on public websites using a vision-language model as a judge [13]. Such judgments can disagree with human assessments, leaving uncertainty about whether a task was completed. To support repeatable evaluation on real websites, we construct VerifiedWebTestBench. Its 197 tasks are adapted from Online-Mind2Web, WebVoyager, and WebGym [2, 13, 42]. For each task, we implement an evaluation script with explicit completion checks over submitted answers and independently collected browser evidence. These checks assess task requirements such as a selected filter, a final page, or a reported fact. Saving the evidence allows results to be inspected and scored again without rerunning the agent. The same checks and saved evidence yield repeatable results across agent interfaces. Using these shared checks, we compare task success and API cost for five agent systems across four LLM backbones on the same collection. The results show that CAE Agent maintains high task success across all four backbones, with a success-rate spread of only 3.55 percentage points. Compared with AgentBrowser [38], CAE Agent achieves slightly higher success on every backbone while nearly halving aggregate API cost. These results demonstrate economical execution with consistently high success across the evaluated models.

Our contributions are as follows.

Table 1 Information exposed by web-agent interfaces. The Targets column indicates whether observations include explicit target references. ✓ explicit, △ implementation-dependent or optional, × not exposed.

Interface	Pixels	Structure	Targets	Actions	Effects
Screenshot [14]	✓	×	×	×	×
Screenshot + marks [31]	✓	△	✓	×	×
DOM / HTML [47, 51]	×	✓	△	×	×
Accessibility tree [18, 51]	×	✓	△	△	×
Pruned structure [18, 47]	×	✓	△	×	×
Hybrid [23]	✓	✓	△	△	×
CAE	×	✓	✓	✓	✓

- Browser-native representation. We introduce CAE, which connects page Content, available Actions, and observed Effects in a structured interaction interface.
- Agent design. We design CAE Agent to use CAE through incremental updates and observed Effects, reducing interaction cost while maintaining high task success.
- Benchmark and evaluation. We construct VerifiedWebTestBench with 197 real-website tasks and explicit checks over saved evidence. We evaluate five agent systems across four LLM backbones and compare representation sizes on matched browser states.

2 Related Work

2.1 Web Agent Observation Interfaces

Page content and structure provide the starting point for web observations. WebArena exposes accessibility-tree observations, while Mind2Web uses cleaned HTML for action prediction [8, 51]. WebFormer, Structor, and MarkupLM encode relationships between text and markup for document understanding [25, 39, 48]. HtmlRAG preserves useful HTML structure during retrieval [37]. Pix2Struct learns screenshot representations through screenshot-to-HTML pretraining [24]. Interactive use additionally requires locating actionable targets. CogAgent, OmniParser, SeeAct, and Dual-VCR use visual grounding, marked regions, or visual–DOM associations for this purpose [14, 19, 31, 50]. Target identifiers specify where an action can be applied, while per-target operation lists specify which actions are available. Compact observations must retain the information needed for these interactions. AgentOccam simplifies observation and action spaces together [43]. DOM pruning, line selection, and snapshot downsampling reduce the material presented at each decision [17, 18, 35, 47]. Accessibility restructuring and region-based organization preserve useful groupings within these smaller views [22, 36].

After acting, an agent needs information about the outcome. BrowserGym includes action-error feedback in its observations [23]. Agent-E uses change observations to guide subsequent decisions, and Tactile provides verification cues [1, 27].

The resulting design question is how compact content, target-specific operations, and observed outcomes fit together throughout interaction. CAE connects them through browser rendering state and execution events, without a language-model call during page construction. Table 1 compares the information exposed by representative interfaces. The Actions column indicates whether operations are listed for each target. The Effects column indicates whether classified browser outcomes are linked to dispatched actions.

2.2 Web Test Datasets and Evaluation

Controlled environments make task completion directly observable. MiniWoB++ and WebShop define assessable tasks within constrained web environments [26, 44]. WebArena and VisualWebArena extend this approach to functional, self-hosted websites [21, 51], while enterprise workflows and personalization broaden the task scope [3, 6, 9]. REAL combines state checks with model judgments in deterministic website simulations [10]. Control over environment state supports explicit completion checks, but these settings do not

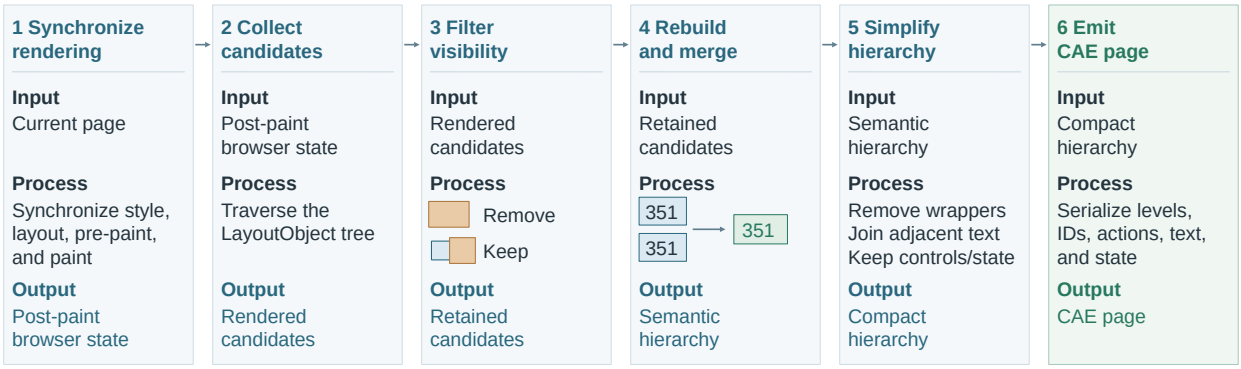


Figure 2 Construction of a CAE page. The six stages show their inputs, processing, and outputs. In visibility filtering, orange boxes cover blue candidates. Fully covered candidates are removed, while partially covered candidates remain. Matching IDs illustrate fragments merged into one semantic node.

capture the full variability of public websites. Public-website benchmarks trade that control for realistic interaction. Mind2Web and WebLINX evaluate action prediction against recorded demonstrations [8, 29]. WebVoyager and Online-Mind2Web assess task outcomes with model-based judges [13, 42]. Online-Mind2Web's comparison with human judgments shows that evaluator choice can affect the measured result. WebGym and WebCanvas express requirements through rubrics and intermediate milestones [2, 34]. AgenticShop evaluates personalized product curation on the open web using a checklist-based framework [20]. Repeatable assessment further requires an executable mapping from collected evidence to a completion verdict.

VerifiedWebTestBench makes this mapping part of each real-website task. Its evaluation scripts apply explicit completion checks to submitted answers and independently collected browser evidence. Separating evidence getters from evaluation functions follows the approach exemplified by OSWorld [41]. Saving these inputs allows the same checks to be repeated offline across agent interfaces, while validation of check correctness and coverage addresses concerns raised by AgentRewardBench [30]. This provides a shared, inspectable basis for the task-success comparisons in our evaluation.

2.3 Agent Harnesses and Interaction Cost

Agent harnesses determine how observations enter successive model decisions. ReAct interleaves reasoning and actions, and WebAgent combines planning with HTML summarization [12, 45]. Workflow memory and learned skills reuse action sequences across tasks [40, 49]. Within a task, the harness must also decide how much page state and history to include as observations accumulate. Browser interfaces offer concrete mechanisms for controlling this context. The agent-browser CLI exposes accessibility snapshots with references for targeted actions, alongside compact views and snapshot differences [38]. The harness determines how these observations and earlier actions enter the next request. Request construction also determines which computation can be reused. Prompt Cache reuses attention states for recurring prompt segments [11]. For browser interaction, stable instructions and earlier context can form a shared request prefix as new actions and observations are appended. Cost-aware evaluation assesses task success together with the resources used to achieve it [15, 16, 28].

CAE Agent maintains a versioned page view through incremental updates and observed Effects. Its context builder appends small page patches after stable instructions and earlier context, preserving a reusable request prefix. Our evaluation compares complete agent configurations across LLM backbones using both task success and API cost.

3 The CAE Interface

A CAE page is a compact hierarchy of rendered text and controls with identifiers, supported operations, and available state. Document structure alone does not establish whether a target is covered or which rendered

fragments belong to the same control, so CAE starts from the browser's post-paint state. We first define the page format, then describe its construction and the Action and Effect interfaces. Figure 1 illustrates the format. Controls use `[LEVEL|ID|ACTIONS]TEXT`. Text rows use `[LEVEL]TEXT`. Optional metadata records values and states derived from browser fields and control-type rules. Omitted attributes remain unspecified. The following example groups a City field and Search button.

```
[2]Search flights
  [3|41|input]{value=""} City
  [3|42|click]Search
```

To fill the City field, the agent calls `input(41, "Paris")`.

3.1 Page Representation

Synchronizing rendering and collecting candidates

After synchronizing the main frame's style, layout, pre-paint, and paint stages, we traverse Blink's `LayoutObject` tree to collect rendered text, geometry, semantic roles, control state, and supported operations (Figure 2). Page construction requires no language-model calls and covers rendered content across the scrollable page, including content below the viewport. Unrendered items in lazy-loaded or virtualized lists are absent until interaction causes them to render.

Each candidate has visible rectangles in page coordinates. For text, each line fragment has its own rectangle, excluding the whitespace between lines. Blink's visual-rectangle mapping accounts for transforms, clipping, and overflow. We sort candidates by `PaintLayer`, using DOM order to break ties within a layer.

Conservative visibility filtering

The visibility test removes a candidate only when reliably opaque boxes painted later cover all its visible rectangles. Objects from the same host element and its descendants are excluded from the test. We round candidate rectangles outward and covering rectangles inward to avoid overestimating coverage. Candidates with uncertain geometry, translucency, unsafe animation, masks, or clip paths remain, as do partially covered candidates.

Restoring structure and merging contributions

Filtering precedes merging because fragments of one control can differ in visibility. After filtering, we restore DOM order and attach children of removed wrappers to their nearest retained ancestor. Objects with the same host identifier merge into one semantic node with combined geometry and deduplicated operations. For native input, textarea, and select controls, browser-generated shadow-tree objects map to the host control. Other shadow-tree structures retain their hierarchy. Each exported control thus retains one identity.

Simplifying the representation

A traversal from leaves to root removes redundant wrappers, repeated labels, and empty shells, and joins adjacent text. It preserves table–row–cell relations, meaningful semantic nodes, operations, and available form state. Folded control rows retain their combined operations and target mappings.

3.2 Actions and Effects

Action takes a target identifier, an advertised operation, and arguments. Supported operations include click, hover, text input, input-and-submit, key press, and selection. Before dispatch, the executor checks page readiness, target validity, supported operations, and arguments. The readiness check uses document progress, network activity, and DOM stability, with a bounded fallback once network activity subsides. Identifiers belong to the current document. After document replacement or substantial reconstruction, the agent must read fresh Content to obtain current targets. Stale or unsupported operations are rejected.

Effect combines renderer observations with browser-process events. The renderer compares control values, focus, the URL, and semantic content before and after an operation. Navigation can replace the originating

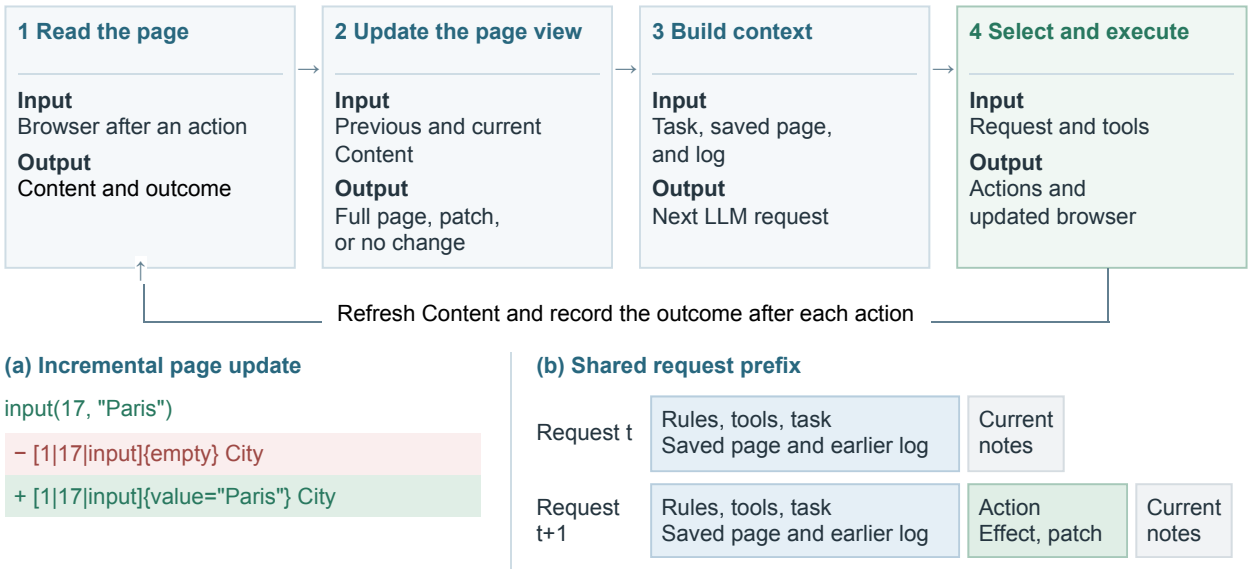


Figure 3 The CAE Agent harness. After each action, the executor refreshes Content and records the returned Effect or page-level receipt. For a small change on the same page, the harness retains the saved page and appends a patch to the log. Consecutive requests preserve a shared prefix that providers may cache, followed by the new action, Effect, and patch.

Table 2 Effect types and reported page states. States describe observations within the bounded reporting window.

Effect type	Observation reported
`navigation`	Navigation is observed.
`new_target`	A new browser target opens.
`same_page_mutation`	Content changes within the same page.
`focus_changed`	Focus moves between elements.
`value_changed`	A control's value changes.
`download`	A download event is observed.
`no_op`	No visible change is observed.
`failed`	The operation reports failure.
Page state	**Interpretation**
`settled`	The observed transition has settled.
`loading`	Loading remains in progress.
`blocked`	Page progress is reported as blocked.
`timeout`	The observation reports a timeout.
`unknown`	Page state could not be established.

renderer, so browser-process events provide continuity across document replacement and capture new targets and downloads. Table 2 lists the Effect vocabulary and page states. Reports can contain multiple changes. The notation `navigation(settled)` in Figure 1 pairs a navigation Effect with its reported page state.

A per-session lock spans action execution, one-time Effect collection, and the subsequent Content refresh. It prevents another action from changing the baseline between these steps. This serializes actions within a session, so the next action waits for observation and refresh. Independent sessions can proceed concurrently.

The observation window is bounded. A `no_op` report therefore does not exclude a later asynchronous change. CAE Agent cancels queued actions after no visible effect and returns to the decision loop (Section 4.1). Node operations return structured Effects, while page-level navigation, tab switching, and waiting use their own receipts. Task completion is assessed independently (Section 5).

4 The CAE Agent

CAE Agent connects an LLM to CAE through a versioned page view, an ordered action executor, and a context builder (Figure 3). The LLM selects operations and supplies arguments.

4.1 Closed-Loop Execution

The harness reads a full Content outline and derives tool schemas from advertised node operations and page-level capabilities. The LLM returns one or more function calls. The executor dispatches them in order and expands batched text entry into individual actions. After each action, the executor refreshes Content and records the returned Effect or page-level receipt, together with the target and arguments. The executor cancels queued actions after navigation, a new target, failure, an unavailable Effect report, or no visible change. Later calls may depend on a state change that did not occur, even if their target identifiers still exist. Returning to the decision loop lets the LLM use refreshed Content and observed Effects to revise the remaining actions. The next decision context includes repetition and stall warnings. Before submission, the harness asks the LLM to review its answer against the task requirements. Independent benchmark checks determine success.

4.2 Incremental State and Prefix Reuse

The browser returns a complete outline after each action. The harness compares successive outlines line by line and encodes small changes as versioned patches. Each patch marks removed lines with `-` and inserted lines with `+`, using nearby context to locate the edit. For an unchanged outline, the log records the action and Effect. The harness replaces the baseline with a full outline after navigation, an active-tab change, or a change in available operation types. It also replaces the baseline when a patch exceeds 30 lines or accumulated patches contain more characters than the new outline. Updates preserve node identifiers, operations, and state text without a model summarizer.

The context builder orders the user message as task, versioned baseline, chronological action/Effect/patch log, and current results and warnings. Ordinary steps extend the log. Fixed instructions, stable tool schemas, and unchanged earlier context preserve a common request prefix. Placing frequently changing information last preserves more of the prefix. Baseline replacement, tool changes, and recovery-note edits can shorten the prefix. This layout supports within-task caching.

5 VerifiedWebTestBench

Each VerifiedWebTestBench task defines explicit completion checks on a real website. Tasks are adapted from Online-Mind2Web [8, 42], WebVoyager [13], and WebGym [2]. We use the same adapted tasks to evaluate agents using CAE and other interfaces.

5.1 Evaluation Model

Model-based judges map a task and selected observations to a verdict through a judge model and a prompt. Their decision rule is not explicit, and changing the judge version can change the verdict. In VerifiedWebTestBench, a specification τ assigns each requirement a result getter g_k, a reference getter $g_k^\star$, and a metric m_k. Given an episode h, including browser evidence and the submitted answer,

$$e_k = g_k(h), \quad r_k = g_k^\star(h), \quad c_k = m_k(e_k, r_k). \tag{1}$$

The metric compares the collected result e_k with its reference r_k and returns a score $c_k \in [0, 1]$. References can be constants or observations collected during the same episode. The aggregation rule A_τ combines these scores into the task result,

$$\widehat{y}_{\text{func}} = \mathbf{1}[A_\tau(c_1, \ldots, c_K) \geq 1]\,. \tag{2}$$

The default conjunction requires every check to pass. Dependencies specify prerequisite checks. With fixed evidence and a fixed specification, the evaluator produces the same score without a judge-model call.

5.2 Task Construction

Human annotators inspect the live site, clarify the instruction, and decompose it into observable requirements. Each task retains its source benchmark, original identifier, and adaptation rationale. For every requirement, human annotators select a result getter, a reference, and a metric. Site-specific selectors and expected conditions reside in the task configuration, while getters and metrics are reused across tasks.

The following construction example extends a television browsing task with an answer requirement. It uses abbreviated pseudocode to show the checks and their dependencies.

```
Instruction: View the most popular TV shows
  and report the top three.
Checks:
  c1: final_url -> url_pattern_match
      ref: popular-TV page URL
  c2: control_state -> json_object_match
      ref: sort == "popularity.desc"
  c3: agent_answer -> answer_set_match
      ref: dom_extract(".card h2")[0:3]
Dependencies: c2 requires c1
              c3 requires c1, c2
Aggregation: all checks pass
```

The reference for c3 comes from the page observed in the same episode, so the answer check follows the displayed ranking as it changes. The URL and sorting checks establish which list supplies that reference. Saving both the answer and the extracted reference allows the same comparison to be repeated offline.

5.3 Validation

We test each evaluation script with correct completions and examples that violate task requirements, such as a wrong entity, a missing filter, or an incomplete answer. These tests check that the script accepts correct results and rejects incorrect ones. We also test the scripts with missing or malformed evidence to check whether they detect collection errors. We verify task feasibility through independent executions in which the agent cannot access the evaluation script. We review submitted answers against each task requirement to check the coverage of the evaluation script. Ambiguous requirements or answer statements that the script cannot assess lead us to revise the task specification. We retain these validation examples with each task.

For tasks that depend on interaction history, we collect evidence during execution. We save this evidence and the submitted answer for offline scoring, keeping evaluation separate from the agent's decisions. The saved inputs allow scores to be checked again without rerunning the agent. Task identifiers, instructions, specifications, and content hashes record which task versions were evaluated.

5.4 Statistics

VerifiedWebTestBench contains 197 distinct source tasks, 129 starting hostnames, and 548 checks in its task specifications (Table 3). The median number of checks per task is 3, with a range of 1–9. Per-task validation records document coverage. Our experiments cover all 197 tasks (Section 6). Table 4 summarizes the evidence used.

Figure 4 groups the tasks into nine website categories. We combine source labels with manual classification of starting websites. Each task belongs to one category based on its starting site, regardless of its answer topic or later destinations.

Of the 197 tasks, 97 use browser evidence alone, 60 combine answers with browser evidence, and 40 compare submitted answers with static references. History-dependent tasks collect evidence during execution. The collection uses 16 non-constant getter types and 27 metric types. Appendix A lists the getter and metric interfaces.

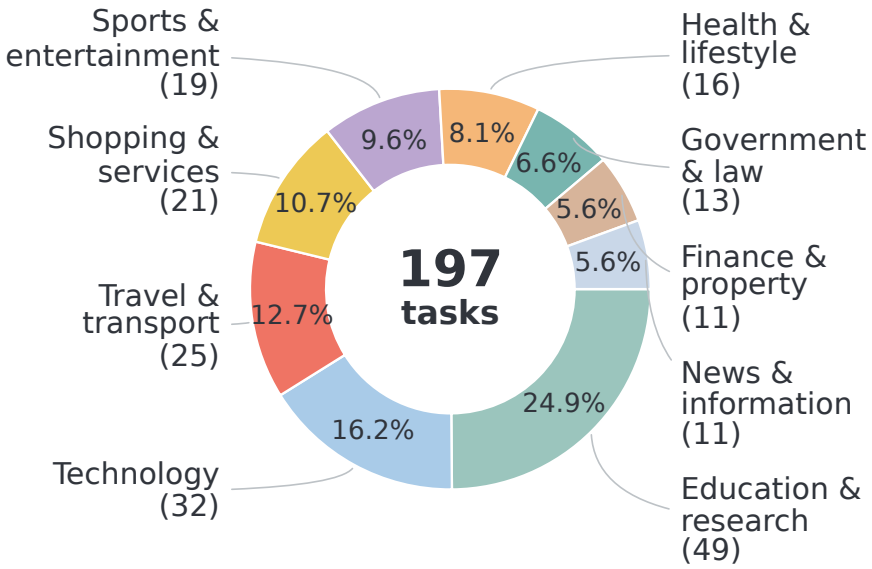


Figure 4 Website-category distribution of all 197 VerifiedWebTestBench tasks. Each task is counted once by its starting website. Labels give task counts and sectors show their shares of the dataset.

Table 3 Composition of VerifiedWebTestBench. Online-Mind2Web and WebVoyager share four start hosts (apple.com, coursera.org, arxiv.org, and espn.com), so the 133 source-level host counts correspond to 129 unique hosts.

Task source	Tasks	Start hosts	Checks
Online-Mind2Web	81	52	229
WebVoyager	43	8	107
WebGym	73	73	212
Total (unique)	197	129	548

6 Experiments

We evaluate four LLM backbones and five agent systems on all 197 tasks in VerifiedWebTestBench. We measure task success, model calls, API cost, and controller elapsed time. A separate comparison measures character and token counts of native CAE, Browser Use DOM, and AgentOccam representations from matched browser states.

6.1 Experimental Setup

Every configuration uses the same 197-task collection. Kimi-k3 [32], GLM-5.3 [46], GPT-6.1-sol [33], and DeepSeek-v4.1-flash [7] are each paired with Browser Use, AgentBrowser, CAE Agent, Jev Ultrafast, and AgentOccam.

All five systems use text observations in this evaluation. Browser Use [4] reads an indexed DOM representation and exposes element actions, navigation, scrolling, and tab controls. Its native decision loop maintains memory and a next-step goal alongside action generation. AgentBrowser [38] reads an accessibility-tree snapshot with target references and executes typed element, navigation, and tab actions. For AgentBrowser, a function-calling controller selects one action per turn, without a separate planner. CAE Agent reads a CAE page with target identifiers, available Actions, and observed Effects. Its LLM selects node operations and page-level navigation or tab actions from versioned decision contexts, without a separate planner. Jev Ultrafast [5] reads a DOM-derived snapshot of visible targets. Its Jev policy selects among click, type, select, scroll up/down, wait, done, and blocked, with no navigation, search, or back action and no separate planner.

Table 4 Nonexclusive evidence groups in VerifiedWebTestBench. A task is counted if a check declares a corresponding getter.

Evidence group	Tasks
URL or open tabs	122
Page content or control state	129
Storage or API state	3
Interaction history	17
Submitted answer	100

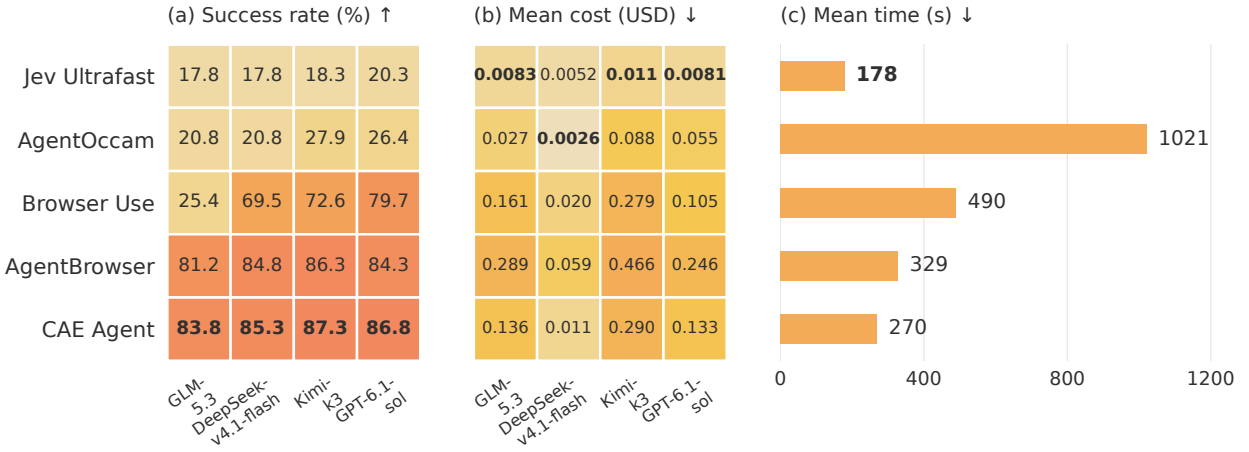


Figure 5 Success and mean cost per model–agent configuration (197 tasks per cell). Cost colors use a logarithmic scale. Time bars average all four backbones (788 task executions per agent system). Bold values mark the highest success and lowest cost per model, and the lowest mean time.

The named backbone supplies field text and the final answer, while `typesafe/jev-1.13` selects actions. AgentOccam [43] reads a pruned accessibility tree and uses its native text commands for clicking, typing, scrolling, back/home navigation, notes, and stopping. Its PlanTree maintains subplans through branch and prune commands within the native decision loop.

Task specifications, independent evidence collection, and functional scoring are shared. Each configuration has a budget of 30 steps, counted according to its native step definition, and a 4,500-second outer watchdog. Available actions differ.

Success rates use all 197 tasks per configuration. Cost, token, call, and time means use the same selected executions. API cost combines recorded fees with frozen-tariff estimates where needed. Tokens and costs use within-episode response-ID deduplication. Controller time covers each execution from setup through cleanup.

6.2 Task Success and Cost

Across the 20 configurations, CAE Agent combines high task success, little variation across backbones, and substantially lower API cost than the similarly successful AgentBrowser. Figure 5 compares success rates, mean API costs, and mean elapsed times. AgentBrowser is accurate and stable but more expensive, Browser Use varies widely across backbones, and Jev Ultrafast and AgentOccam have low costs but low task success.

CAE Agent exceeds 83% success on every backbone and reaches 85.79% in aggregate, compared with AgentBrowser's 84.14%. Every Jev Ultrafast and AgentOccam configuration falls below the 47.72% human success rate (Section 6.4).

The gap between the highest and lowest success rates across backbones is 3.55 percentage points for CAE Agent, 5.08 for AgentBrowser, and 54.31 for Browser Use. We argue that listing each target's available

Table 5 Mean page-representation size on 167 matched pages, using native outputs. Relative to full-page DOM, CAE uses 31.7% fewer characters and 26.7% fewer tokens. Reduction columns use aggregate counts. All token counts use `o200k_base` and exclude system prompts and tool descriptions. AgentOccam retains its native 1,000-record limit. Negative reductions indicate larger outputs.

Representation	Mean size		Reduction	
	Chars.	Tokens	Chars.	Tokens
Browser Use DOM (full page)	12,667	4,171	0.0%	0.0%
AgentOccam (pruned AX)	17,565	5,717	-38.7%	-37.1%
CAE	8,653	3,056	**31.7%**	**26.7%**

Actions in Content reduces the need to infer applicable operations and their syntax, making action selection easier for weaker backbones.

Compared with AgentBrowser, CAE Agent achieves slightly higher success on every backbone with 46.18% lower aggregate API cost. Per-backbone savings range from 37.66% to 81.88%. Dividing total cost, including the cost of failed tasks, by the number of successful tasks gives $0.166 per success for CAE Agent and $0.315 for AgentBrowser, a 47.22% reduction. Together, these results show high task success across backbones with substantially lower cost than AgentBrowser.

6.3 Page-Representation Size

We compare native CAE, Browser Use DOM, and AgentOccam representations on 167 entry pages from WebGym, WebVoyager, and Online-Mind2Web, capturing all three from the same browser state. Because CAE covers rendered content across the full scrollable page, we disable Browser Use's viewport clipping to match this coverage. We retain native formatting and other limits, without additional filtering or truncation.

These measurements cover the captured entry states. On infinite-scroll feeds and virtualized product or result lists, unrendered items remain outside the current CAE page. Scrolling or loading more content requires a fresh observation.

Table 5 reports means across the 167 matched states. CAE averages 8,653 characters and 3,056 tokens, compared with 12,667 characters and 4,171 tokens for full-page DOM and 17,565 characters and 5,717 tokens for AgentOccam. Relative to DOM, the aggregate CAE output contains 31.7% fewer characters and 26.7% fewer tokens.

6.4 Human Performance on VerifiedWebTestBench

Across two rounds, 20 participants made 202 submissions covering all 197 tasks in VerifiedWebTestBench. We retain the earliest submission per task, excluding five duplicates. Table 6 reports 94 successes and 103 failures (47.72% success). Across all 197 tasks, server-recorded time from browser readiness to submission averaged 175.04 s (median 121.60 s). Task completion is assessed using evaluation scripts and review of archived evidence and participant reports.

Reviewing archived human submissions reveals several concrete failure patterns, including incomplete answers, incorrect output units, and stopping on a related page rather than the requested one. For example, one participant named the requested baseball team but omitted the total number of teams; successful agents supplied both. In another task, the participant stopped on a statistics page, whereas a successful agent reached the requested standings page. These examples illustrate the need to track all task requirements, locate the relevant information, and check that the final submission covers the requested result.

Table 6 Human performance on all 197 tasks, using the earliest submission per task and mixed outcome sources.

Measure	Human
Participants	20
Successful / analyzed submissions	94/197
Task success rate (%)	47.72
Mean time per task (s)	175.04
Median time per task (s)	121.60

7 Conclusion

We introduced CAE, a browser-native representation that connects page Content, available Actions, and observed Effects. By organizing rendered content around identified targets and supported operations, CAE links each action to observed changes in page state. Across 167 matched entry pages, CAE reduces aggregate token count by 26.7% compared with full-page Browser Use DOM outputs. Building on this representation, we designed CAE Agent, an LLM-based web agent that maintains a versioned page view and constructs decision contexts from incremental updates and observed Effects.

To support repeatable evaluation on real websites, we constructed VerifiedWebTestBench with 197 tasks and explicit completion checks over independently collected evidence. Across five agent systems and four LLM backbones, CAE Agent maintains success above 83% on every backbone, with a spread of only 3.55 percentage points. CAE Agent achieves slightly higher task success than AgentBrowser on every backbone while reducing aggregate API cost by 46.18%. These results show that CAE Agent combines consistently high task success across the evaluated models with API cost savings that hold across all four backbones.

References

[1] Tamer Abuelsaad, Deepak Akkil, Prasenjit Dey, Ashish Jagmohan, Aditya Vempaty, and Ravi Kokku. 2024. Agent-E: From Autonomous Web Navigation to Foundational Design Principles in Agentic Systems. *arXiv preprint arXiv:2407.13032* (2024). https://arxiv.org/abs/2407.13032

[2] Hao Bai, Alexey Taymanov, Tong Zhang, Aviral Kumar, and Spencer Whitehead. 2026. WebGym: Scaling Training Environments for Long-Horizon Visual Web Agents with Realistic Tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 12248–12258.

[3] Léo Boisvert, Megh Thakkar, Maxime Gasse, Massimo Caccia, Thibault Le Sellier De Chezelles, Quentin Cappart, Nicolas Chapados, Alexandre Lacoste, and Alexandre Drouin. 2024. WorkArena++: Towards Compositional Planning and Reasoning-based Common Knowledge Work Tasks. In *Advances in Neural Information Processing Systems*, Vol. 37. https://proceedings.neurips.cc/paper_files/paper/2024/hash/0b82662b6c32e887bb252a74d8cb2d5e-Abstract-Datasets_and_Benchmarks_Track.html

[4] Browser Use. 2026. Browser Use: The AI Browser Agent. https://github.com/browser-use/browser-use Software repository.

[5] Browser Use. 2026. Jev Ultrafast. https://github.com/browser-use/jev-ultrafast

[6] Hongru Cai, Yongqi Li, Wenjie Wang, Fengbin Zhu, Xiaoyu Shen, Wenjie Li, and Tat-Seng Chua. 2025. Large Language Models Empowered Personalized Web Agents. In *Proceedings of the ACM on Web Conference 2025*. Association for Computing Machinery, 198–215. doi:10.1145/3696410.3714842

[7] DeepSeek. 2026. Introducing DeepSeek-V4.1-Flash: Smarter, Faster, More Efficient. https://www.deepseek.com/en/news/deepseek-v4-1-flash/

[8] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2Web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems* 36 (2023), 28091–28114.

[9] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, et al. 2024. WorkArena: How capable are web agents at solving common knowledge work tasks? *arXiv preprint arXiv:2403.07718* (2024).

[10] Div Garg, Diego Caples, Andis Draguns, Nikil Ravi, Pranav Putta, Naman Garg, Prannay Hebbar, Youngchul Joo, Jindong Gu, Charles London, et al. 2026. REAL: Benchmarking autonomous agents on deterministic simulations of real websites. *Advances in Neural Information Processing Systems* 38 (2026).

[11] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In *Proceedings of Machine Learning and Systems*, Vol. 6. https://proceedings.mlsys.org/paper_files/paper/2024/hash/a66caa1703fe34705a4368c3014c1966-Abstract-Conference.html

[12] Izzeddin Gur, Hiroki Furuta, Austin Huang, Mustafa Safdari, Yutaka Matsuo, Douglas Eck, and Aleksandra Faust. 2024. A Real-World WebAgent with Planning, Long Context Understanding, and Program Synthesis. In *The Twelfth International Conference on Learning Representations*. arXiv:2307.12856 https://openreview.net/forum?id=9JQtrumvg8

[13] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. 2024. WebVoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 6864–6890.

[14] Wenyi Hong, Weihan Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. 2024. CogAgent: A visual language model for GUI agents. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 14281–14290.

[15] Sayash Kapoor, Benedikt Stroebl, Peter Kirgis, Nitya Nadgir, Zachary Siegel, Boyi Wei, Tianci Xue, Ziru Chen, Felix Chen, Saiteja Utpala, et al. 2026. Holistic agent leaderboard: The missing infrastructure for AI agent evaluation. In *International Conference on Learning Representations*, Vol. 2026. 98778–98849.

[16] Sayash Kapoor, Benedikt Stroebl, Zachary S Siegel, Nitya Nadgir, and Arvind Narayanan. 2024. AI agents that matter. *arXiv preprint arXiv:2407.01502* (2024).

[17] Imene Kerboua, Sahar Omidi Shayegan, Megh Thakkar, Xing Han Lù, Massimo Caccia, Véronique Eglin, Alexandre Aussem, Jérémy Espinas, and Alexandre Lacoste. 2025. LineRetriever: Planning-Aware Observation Reduction for Web Agents. arXiv:2507.00210 https://arxiv.org/abs/2507.00210

[18] Imene Kerboua, Sahar Omidi Shayegan, Megh Thakkar, Xing Han Lù, Léo Boisvert, Massimo Caccia, Jérémy Espinas, Alexandre Aussem, Véronique Eglin, and Alexandre Lacoste. 2025. FocusAgent: Simple yet effective ways of trimming the large context of web agents. *arXiv preprint arXiv:2510.03204* (2025).

[19] Jihyung Kil, Chan Hee Song, Boyuan Zheng, Xiang Deng, Yu Su, and Wei-Lun Chao. 2024. Dual-view visual

contextualization for web navigation. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 14445–14454.

[20] Sunghwan Kim, Ryang Heo, Yongsik Seo, Jinyoung Yeo, and Dongha Lee. 2026. AgenticShop: Benchmarking Agentic Product Curation for Personalized Web Shopping. In *Proceedings of the ACM Web Conference 2026*. Association for Computing Machinery. doi:10.1145/3774904.3792724

[21] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. 2024. VisualWebArena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 881–905.

[22] Donguk Kwon and Dongha Lee. 2026. Region4Web: Rethinking Observation Space Granularity for Web Agents. *arXiv preprint arXiv:2605.07134* (2026).

[23] Thibault Le Sellier De Chezelles, Maxime Gasse, Alexandre Drouin, Massimo Caccia, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omidi Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Quentin Cappart, Graham Neubig, Ruslan Salakhutdinov, Nicolas Chapados, and Alexandre Lacoste. 2024. The BrowserGym Ecosystem for Web Agent Research. *arXiv preprint arXiv:2412.05467* (2024). https://arxiv.org/abs/2412.05467

[24] Kenton Lee, Mandar Joshi, Iulia Raluca Turc, Hexiang Hu, Fangyu Liu, Julian Martin Eisenschlos, Urvashi Khandelwal, Peter Shaw, Ming-Wei Chang, and Kristina Toutanova. 2023. Pix2Struct: Screenshot Parsing as Pretraining for Visual Language Understanding. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 18893–18912. https://proceedings.mlr.press/v202/lee23g.html

[25] Junlong Li, Yiheng Xu, Lei Cui, and Furu Wei. 2022. MarkupLM: Pre-training of Text and Markup Language for Visually Rich Document Understanding. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 6078–6087. doi:10.18653/v1/2022.acl-long.420

[26] Evan Zheran Liu, Kelvin Guu, Panupong Pasupat, Tianlin Shi, and Percy Liang. 2018. Reinforcement learning on web interfaces using workflow-guided exploration. *arXiv preprint arXiv:1802.08802* (2018).

[27] Yong Liu, Zhenyi Zhong, and Zhanpeng Shi. 2026. Tactile: Giving Computer-Using Agents Hands and Feet. *arXiv preprint arXiv:2607.14443* (2026).

[28] Renze Lou, Baolin Peng, Wenlin Yao, Qianhui Wu, Hao Cheng, Suman Nath, Wenpeng Yin, and Jianfeng Gao. 2026. The Tool Illusion: Rethinking Tool Use in Web Agents. In *Conference on Language Modeling*. arXiv:2604.03465 https://arxiv.org/abs/2604.03465

[29] Xing Han Lu, Zdeněk Kasner, and Siva Reddy. 2024. WebLINX: Real-World Website Navigation with Multi-Turn Dialogue. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 33007–33056. https://proceedings.mlr.press/v235/lu24e.html

[30] Xing Han Lù, Amirhossein Kazemnejad, Nicholas Meade, Arkil Patel, Dongchan Shin, Alejandra Zambrano, Karolina Stańczak, Peter Shaw, Christopher J Pal, and Siva Reddy. 2025. AgentRewardBench: Evaluating automatic evaluations of web agent trajectories. *arXiv preprint arXiv:2504.08942* (2025).

[31] Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. 2024. OmniParser for pure vision based GUI agent. *arXiv preprint arXiv:2408.00203* (2024).

[32] Moonshot AI. 2026. Kimi K3: Open Frontier Intelligence. https://www.kimi.ai/blog/kimi-k3

[33] OpenAI. 2026. Introducing GPT-6.1 Sol. https://openai.com/index/introducing-gpt-6-1-sol/

[34] Yichen Pan, Dehan Kong, Sida Zhou, Cheng Cui, Yifei Leng, Bing Jiang, Hangyu Liu, Yanyi Shang, Shuyan Zhou, Tongshuang Wu, et al. 2024. WebCanvas: Benchmarking web agents in online environments. *arXiv preprint arXiv:2406.12373* (2024).

[35] Thassilo M. Schiepanski and Nicholas Piël. 2025. Beyond Pixels: Exploring DOM Downsampling for LLM-Based Web Agents. arXiv:2508.04412 https://arxiv.org/abs/2508.04412

[36] Michito Takeshita, Takuro Kawada, Takumi Ohashi, Shunsuke Kitada, and Hitoshi Iyatomi. 2026. A11y-Compressor: A Framework for Enhancing the Efficiency of GUI Agent Observations through Visual Context Reconstruction and Redundancy Reduction. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL 2026)*. 563–580.

[37] Jiejun Tan, Zhicheng Dou, Wen Wang, Mang Wang, Weipeng Chen, and Ji-Rong Wen. 2025. HtmlRAG: HTML is Better Than Plain Text for Modeling Retrieved Knowledge in RAG Systems. In *Proceedings of the ACM on Web Conference 2025*. Association for Computing Machinery, 1733–1746. doi:10.1145/3696410.3714546

[38] Vercel Labs. 2026. agent-browser: Browser Automation CLI for AI Agents. https://github.com/vercel-labs/agent-browser

[39] Qifan Wang, Yi Fang, Anirudh Ravula, Fuli Feng, Xiaojun Quan, and Dongfang Liu. 2022. WebFormer: The Web-page Transformer for Structure Information Extraction. In *Proceedings of the ACM Web Conference 2022*. Association for Computing Machinery, 3124–3133. doi:10.1145/3485447.3512032

[40] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. Agent Workflow Memory. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 63897–63911. https://proceedings.mlr.press/v267/wang25bx.html

[41] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems*, Vol. 37. https://papers.neurips.cc/paper_files/paper/2024/hash/5d413e48f84dc61244b6be550f1cd8f5-Abstract-Datasets_and_Benchmarks_Track.html

[42] Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. 2025. An illusion of progress? assessing the current state of web agents. *arXiv preprint arXiv:2504.01382* (2025).

[43] Ke Yang, Yao Liu, Sapana Chaudhary, Rasool Fakoor, Pratik A Chaudhari, George Karypis, and Huzefa Rangwala. 2025. AgentOccam: A simple yet strong baseline for LLM-based web agents. In *International Conference on Learning Representations*, Vol. 2025. 97533–97565.

[44] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. In *Advances in Neural Information Processing Systems*, Vol. 35. https://papers.neurips.cc/paper_files/paper/2022/hash/82ad13ec01f9fe44c01cb91814fd7b8c-Abstract-Conference.html

[45] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*. https://arxiv.org/abs/2210.03629

[46] Z.ai. 2026. GLM-5.3: Frontier Coding with Emergent Cyber Capabilities. https://z.ai/blog/glm-5.3

[47] Jiayuan Zhang, Kaiquan Chen, Zhihao Lu, Enshen Zhou, Qian Yu, and Jing Zhang. 2026. Prune4Web: DOM tree pruning programming for web agent. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 34710–34718.

[48] Zhenyu Zhang, Bowen Yu, Tingwen Liu, Tianyun Liu, Yubin Wang, and Li Guo. 2023. Learning Structural Co-occurrences for Structured Web Data Extraction in Low-Resource Settings. In *Proceedings of the ACM Web Conference 2023*. Association for Computing Machinery, 1683–1692. doi:10.1145/3543507.3583387

[49] Boyuan Zheng, Michael Y Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, et al. 2025. SkillWeaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079* (2025).

[50] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. 2024. GPT-4V(ision) is a generalist web agent, if grounded. *arXiv preprint arXiv:2401.01614* (2024).

[51] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. 2024. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, Vol. 2024. 15585–15606.

A Evaluation Functions

A getter collects evidence. A metric compares that evidence with a reference and returns a score in $[0, 1]$. Tables 7 and 8 list the 17 getter types (including `const`) and 27 metric types referenced by the 197 task specifications. The Tasks column counts distinct tasks. The Refs. column counts declarations, including getters in both result and reference slots and one metric per check. These are specification counts, not runtime call counts.

Table 7 Getters used in the task specifications and returned evidence.

Function	Tasks	Refs.	Evidence returned
Saved records and constants			
`const`	191	505	Any. Return the configured literal value, commonly used as a reference.
`agent_answer`	100	209	Text. Return the submitted answer verbatim. An absent answer is MISSING.
Navigation			
`final_url`	119	124	Text. Read the current page URL at collection time.
`url_parse`	14	14	Object. Parse URL query parameters. Optionally retain repeated values.
`url_path_parse`	1	1	Text/list. Extract URL path segments or one indexed segment.
`open_tabs`	3	6	List. List the URLs and titles of open page targets.
Page content and controls			
`page_text`	28	28	Text. Read document.body.innerText, with an optional character limit.
`page_title`	11	11	Text. Read the current document title.
`dom_extract`	25	49	Object. Extract named text, attributes, or values using configured selectors.
`dom_count`	12	16	Integer. Count elements matching a configured selector.
`control_state`	8	9	Object. Read values, checked/selected states, text, disabled flags, and ARIA attributes.
`visible_text`	4	4	Text. Collect text after CSS visibility and layout checks. This is not pixel-level occlusion testing.
`axtree_query`	2	3	List. Filter accessibility nodes by role/name and return role, name, and value.
Snapshots and history			
`dom_snapshot`	72	94	Object. Extract task-declared facts from URL-matched tabs, retaining target and document identity.
`snapshot_history`	17	18	Object. Return independently sampled snapshots, timestamps, collection errors, and completion status.
Server, network, and browser storage			
`api_state`	1	1	JSON/text. Fetch a configured endpoint from the page context with the declared credentials policy.
`storage`	2	2	Object. Read configured localStorage or sessionStorage keys under an explicit origin constraint.

Table 8 Metrics used in the task specifications and acceptance conditions.

Function	Tasks	Refs.	Condition checked
URLs and tabs			
`url_matches`	22	22	Compare normalized URLs against one or more candidates. Normalization flags are configurable.
`url_pattern_match`	97	110	Apply URL regular expressions with all/any combination.
`url_query_match`	1	1	Compare parsed query values. Conflicting duplicate parameters raise an error.
`tabs_match`	3	3	Require the specified URLs among open tabs, or require equal URL sets.
Text and structured values			
`include_exclude`	31	32	Require every specified inclusion and reject every specified exclusion.
`json_object_match`	125	198	Apply path-to-rule constraints to structured values. Optional partial credit is explicit.
`json_any_match`	5	5	Accept one complete alternative path-to-rule mapping without mixing branches.
`set_match`	1	1	Check set equality, subset, or superset relations.
`count_in_range`	14	19	Check a count or collection length against declared bounds.
Records and selection			
`records_match`	5	6	Require all field conditions to hold in the same record, for any or all selected records.
`selection_match`	1	1	Verify identity and eligibility. Ranked selection also requires a complete reference catalogue.
Answers and factual claims			
`answer_match`	35	49	Check configured content using structured JSON or text matching.
`answer_dated_records_match`	1	1	Parse identity/date assertions and check coverage of the dated reference records.
`answer_boolean_records_match`	1	1	Parse identity-bound Boolean claims, including declared universal assertions.
`answer_quantity_records_match`	1	1	Bind reported values and units to independently collected reference records.
`answer_set_match`	1	1	Check the reported entity set using declared aliases and inclusion/exclusion grammar.
`answer_fact_match`	18	19	Parse subject-bound factual clauses using task-declared facts and language rules.
`answer_relations_match`	19	25	Parse complete relations with multiple slots and declared conditional requirements.
`answer_dynamic_fact_match`	1	1	Bind numeric facts to independent reference evidence before parsing answer claims.
`answer_dynamic_relations_match`	26	26	Bind relation slots to independent reference evidence before checking the answer.
`answer_dynamic_relations_raw_match`	1	1	Use dynamic relation checking while preserving URLs and bypassing shared wrapper removal.
`bounded_claim_rows_match`	3	7	Check declared category, amount, and eligibility claims with a bounded row grammar.
Quantities and mathematics			
`math_value_match`	1	1	Evaluate supported numeric expressions, including complex values, within a tolerance.
`ode_page_match`	1	1	Check independently captured page input equations and displayed general solutions.
Observed history			
`history_sequence_match`	1	1	Bind an output transition to preceding complete inputs, tracking document identity and capture gaps.
`history_presence_match`	13	13	Require evidence of the specified state or event. Incomplete collection cannot establish its absence.
`history_absence_match`	2	2	Reject observed forbidden states. Passing requires declared observation coverage.

Lexmount Team

Chao Bian, Chenrui Shi, Jinghua Guo, Xianghui Sun, Feng Wang, Min Zhang.